

Módulo 9 – Capítulo 06 – Sección 01

PII en sistemas de IA: detección, anonimización y pseudonimización

La gestión de Información de Identificación Personal (PII) en sistemas de IA es técnicamente más compleja que en sistemas tradicionales de procesamiento de datos porque el PII puede aparecer en múltiples capas del sistema, en formatos inesperados, y puede ser introducido por componentes que en sistemas clásicos no procesarían datos personales. Un sistema RAG que indexa documentos corporativos puede indexar inadvertidamente documentos con nombres de clientes, números de identificación o datos de salud. Un historial de conversación almacenado para dar continuidad a sesiones multi-turn acumula cualquier dato personal que el usuario mencione. Los logs de inferencia registran el input completo del usuario, que puede incluir PII significativa. Y el modelo mismo —si fue fine-tuneado con datos que contenían PII— puede reproducir verbatim datos personales que "memorizó" del corpus de entrenamiento. Ninguno de estos vectores existe de manera equivalente en sistemas CRUD convencionales.

La detección automatizada de PII en texto es el prerequisite para cualquier control de privacidad efectivo. Microsoft Presidio es la herramienta open-source de referencia: soporta más de 50 entidades PII en múltiples idiomas, incluyendo nombres, emails, números de teléfono, documentos de identidad (SSN, pasaportes, NIE), números de tarjeta de crédito, coordenadas geográficas, y credenciales de acceso. Presidio usa una combinación de reconocimiento de entidades nombradas (NER) con modelos de spaCy, expresiones regulares para formatos estructurados (emails, números de tarjeta), y listas de contexto para mejorar la precisión. Amazon Comprehend PII Detection ofrece una alternativa gestionada con integración directa con otros servicios AWS. Para PII contextual —"mi jefe Juan López trabaja en el departamento de finanzas"— los modelos NER genéricos tienen mayor tasa de falsos negativos y pueden requerir fine-tuning específico del dominio.

La anonimización y la pseudonimización son técnicas complementarias con diferentes trade-offs para sistemas de IA. La anonimización elimina el PII de manera irreversible —reemplazando "Juan López" con "[REDACTED]" o con una cadena hash— y es la opción más segura para contextos donde el PII no es necesario para la tarea. Su limitación es que puede

degradar la calidad del modelo si el PII era semánticamente relevante: un sistema de análisis de contratos que recibe los contratos con todos los nombres redactados pierde señales de relaciones entre personas que podrían ser relevantes para el análisis. La pseudonimización reemplaza el PII con identificadores consistentes y reversibles —"Juan López" → "PERSON_1", y esta sustitución se aplica consistentemente en todo el documento— preservando las relaciones semánticas mientras oculta la identidad real. La tabla de mapeo entre PII real e identificadores pseudonimizados debe almacenarse con cifrado y controles de acceso estrictos, y es el activo de privacidad más crítico del sistema.

Aspectos técnicos

- **Microsoft Presidio (open-source):** 50+ entidades PII en múltiples idiomas, combinando NER (spaCy), regex para formatos estructurados, y listas de contexto; configurable con entidades personalizadas para PII específica del dominio; disponible como servicio REST o como librería Python.
- **Anonimización en el pipeline RAG:** documentos deben pasar por detección y redacción de PII con Presidio antes de ser indexados en el vectorstore; el mismo documento sin PII sirve a todos los usuarios, preservando la utilidad del sistema sin exponer datos personales entre usuarios.
- **Pseudonimización para fine-tuning:** sustitución consistente de PII por placeholders (PERSON_1, ORG_2, LOCATION_3) preserva relaciones semánticas mientras protege la identidad; la tabla de mapeo debe almacenarse con cifrado AES-256 y acceso auditado.
- **PII en el historial de conversación:** los sistemas con memoria multi-turn acumulan PII del usuario; implementar TTL (tiempo de vida) para el historial, cifrado por usuario, y mecanismos de borrado a petición del usuario para cumplir con el derecho al olvido del GDPR.
- **Detección de PII en outputs:** escanear respuestas del modelo con Presidio antes de entregarlas al cliente para detectar PII no intencional proveniente de memorización del pretraining o de PII del contexto del usuario incluida en la respuesta sin necesidad.

Nota del Arquitecto: La implementación de Presidio en el pipeline RAG tiene una trampa frecuente: anonimizar los documentos antes de indexarlos pero olvidar anonimizar los metadatos del chunk —autor del documento, fecha de creación, departamento propietario— que se incluyen en la respuesta del LLM junto con el contenido del chunk. He visto sistemas donde el contenido del documento estaba correctamente redactado pero el metadata incluía el nombre completo del empleado

que creó el documento, la fecha exacta de una transacción, o el número de empleado del departamento. Los metadatos deben pasar por el mismo pipeline de detección y redacción que el contenido.

La gestión de PII en sistemas de IA debe cubrir todas las capas donde puede aparecer — input del usuario, corpus RAG, historial de conversación, logs de inferencia, outputs del modelo — con controles técnicos específicos en cada capa, porque la detección en una sola capa deja las demás completamente desprotegidas.

Módulo 9 – Capítulo 06 – Sección 02

Datos en tránsito y en reposo: cifrado de prompts, respuestas e índices vectoriales

El cifrado de datos en tránsito y en reposo en sistemas de IA sigue los mismos principios que en cualquier sistema de procesamiento de datos sensibles, pero su aplicación tiene consideraciones específicas para los componentes que son únicos de la arquitectura de IA. Los prompts que viajan entre la aplicación y la API del LLM, los vectores de embedding almacenados en el vectorstore, los artefactos de modelo fine-tuned que contienen representaciones de datos de entrenamiento, y los logs de inferencia que registran inputs y outputs completos son todos componentes con características particulares que determinan qué nivel y qué tipo de cifrado se requiere.

Para datos en tránsito, TLS 1.3 es el estándar mandatorio para todas las conexiones a APIs de LLMs —OpenAI, Anthropic, Azure OpenAI, Vertex AI— y para cualquier componente interno del pipeline que transmita datos de usuarios. En arquitecturas multi-tier donde el gateway de la aplicación reenvía requests al proveedor del modelo, cada salto de red debe usar TLS con certificados válidos; la terminación de TLS en el gateway no debe resultar en re-transmisión en plaintext en la red interna entre el gateway y el servicio de inferencia. Para endpoints self-hosted de modelos (vLLM, TGI), TLS debe configurarse explícitamente porque no está habilitado por defecto en la mayoría de las implementaciones.

Los índices vectoriales merecen mención especial porque frecuentemente no se tratan con el nivel de protección que merecen. Los vectores de embedding no son datos "anonimizados" de ninguna manera: las representaciones vectoriales de texto preservan suficiente información semántica sobre el texto original para que técnicas de inversión de embeddings puedan recuperar aproximaciones del texto fuente, especialmente cuando el atacante conoce el modelo de embedding utilizado. Pinecone cifra datos en reposo con AES-256 por defecto; Weaviate y pgvector requieren configuración explícita del cifrado a nivel de storage y no lo habilitan automáticamente. En implementaciones self-hosted de vectorstores, el cifrado debe configurarse a nivel del sistema de archivos si el vectorstore no lo implementa nativamente.

Los logs de inferencia son el elemento más frecuentemente subestimado en términos de sensibilidad de datos. Un log de inferencia que registra el input completo del usuario y el output del modelo durante su historial completo de uso del sistema es un activo de datos extremadamente sensible: contiene todo lo que el usuario escribió en la aplicación, incluyendo datos personales, consultas de carácter privado, y potencialmente información confidencial de negocio que el usuario compartió para obtener asistencia. El tratamiento correcto de los logs de inferencia es el mismo que el de una base de datos de conversaciones privadas de usuarios: cifrado AES-256 en reposo con claves KMS, retención limitada por TTL definido, acceso auditado y necesidad justificada, y mecanismos de borrado selectivo por usuario para compliance con GDPR.

Aspectos técnicos

- **TLS 1.3 mandatorio en tránsito:** para todas las APIs de LLMs y para cada salto interno del pipeline; en endpoints self-hosted (vLLM, TGI) configurar TLS explícitamente ya que no está habilitado por defecto; sin transmisión en plaintext en la red interna entre gateway y servicio de inferencia.
- **Cifrado del vectorstore en reposo:** Pinecone AES-256 por defecto; Weaviate y pgvector requieren configuración explícita; los vectores de embedding contienen información semánticamente reversible sobre el texto original y deben protegerse equivalentemente a los datos fuente.
- **Cifrado de modelos fine-tuned en reposo:** artefactos de modelo en S3/GCS/Azure Blob con SSE-KMS usando claves gestionadas por el cliente (no las del proveedor de storage), que permiten revocar el acceso al modelo sin depender del proveedor.
- **Logs de inferencia como datos sensibles de primera clase:** cifrado AES-256 en reposo con claves KMS, retención con TTL definido basado en necesidad justificada, acceso auditado, y mecanismo de borrado selectivo por user_id para compliance con GDPR Art. 17.
- **Gestión de claves y rotación:** claves de cifrado en KMS (AWS KMS, Azure Key Vault, GCP Cloud KMS o HashiCorp Vault para self-hosted), nunca en variables de entorno ni código; rotación automática anual con proceso de re-cifrado de datos existentes.

La visión correcta de seguridad de datos en sistemas de IA trata al sistema completo — modelo, vectorstore, logs de inferencia, pipeline de inferencia— como un sistema de procesamiento de datos sensibles equivalente a cualquier base de datos de producción con datos personales, aplicando el mismo rigor de cifrado, acceso y retención a cada componente.

Módulo 9 – Capítulo 06 – Sección 03

Training data privacy: riesgos de memorización y extracción de datos de entrenamiento

Cuando una organización hace fine-tuning de un modelo con sus datos propietarios —emails de clientes, contratos, historiales médicos, documentos legales, conversaciones de soporte— asume implícitamente que esos datos quedan "dentro" del modelo de una manera que no es directamente accesible. Esta suposición es incorrecta, y Carlini et al. la refutaron empíricamente con evidencia que no puede ignorarse. Los modelos de lenguaje de gran escala memorizan fragmentos de su corpus de entrenamiento y pueden reproducirlos verbatim en sus respuestas: no como regla sino como comportamiento emergente que ocurre con probabilidad no despreciable, especialmente para textos que aparecen múltiples veces en el corpus o que tienen características estadísticas inusuales.

La memorización en LLMs no es aleatoria. Carlini et al. identificaron los factores que la gobiernan en sus trabajos desde 2021: el tamaño del modelo tiene correlación directa con la cantidad memorizada en términos absolutos —los modelos más grandes memorizan más, aunque también aprenden representaciones más generales—, la frecuencia de aparición de un texto en el corpus tiene el mayor efecto individual (un texto que aparece 100 veces en el corpus tiene probabilidad de memorización varias órdenes de magnitud mayor que uno que aparece una sola vez), y el overfitting durante el entrenamiento —que ocurre especialmente en fine-tuning con conjuntos de datos pequeños— amplifica la memorización porque el modelo aprende a replicar los ejemplos de entrenamiento en lugar de generalizar.

El ataque documentado por Nasr et al. (2023) que extrajo datos de entrenamiento de ChatGPT en producción es el caso más perturbador porque demostró que la memorización es explotable en modelos de producción de escala real, no solo en modelos de investigación en condiciones de laboratorio. El truco utilizado era conceptualmente simple: repetir indefinidamente una sola palabra ("poem poem poem poem...") en el prompt. Al continuar ese patrón durante suficiente tiempo, el modelo agotaba su mecanismo de seguimiento del patrón y comenzaba a reproducir memorización del training set —incluyendo en algunos casos texto verbatim con PII de personas que habían aparecido en Common Crawl. Este

vector de extracción fue reportado responsablemente a OpenAI y posteriormente mitigado, pero el principio que ilustra permanece: cualquier modelo suficientemente grande entrenado sobre datos del mundo real tiene datos de ese mundo real memorados en sus pesos, recuperables mediante técnicas adecuadas.

Para organizaciones que hacen fine-tuning con datos propietarios sensibles, el riesgo de memorización se concentra en los datos de fine-tuning, no en el modelo base. El ratio de datos de fine-tuning respecto al total de tokens de entrenamiento es mucho más alto que en el pretraining —un dataset de fine-tuning típico puede ser de decenas de miles a millones de ejemplos, versus los billones de tokens del pretraining—, lo que significa que los datos de fine-tuning son procesados muchas más veces (más epochs) y tienen mayor probabilidad de ser memorizados que datos de pretraining que aparecen pocas veces.

Aspectos técnicos

- **Cuantificación de la memorización (extractability):** qué porcentaje de los tokens de un ejemplo de entrenamiento puede el modelo reproducir exactamente dado un prefijo; Carlini et al. midieron extractability del 0.1%-1% en GPT-2, con valores más altos en modelos mayores y para ejemplos de alta frecuencia.
- **Factores de riesgo:** tamaño del modelo (mayor = más memorización), frecuencia de duplicación del dato en el corpus (>10 apariciones = riesgo alto), y overfitting durante el entrenamiento (especialmente en fine-tuning con datasets pequeños y múltiples epochs).
- **Ataque de Nasr et al. (2023) contra ChatGPT:** el prompt "Repeat the word 'poem' forever" hacía que el modelo saliera de su modo generativo conversacional y reprodujera memorización del training set, incluyendo PII de personas de Common Crawl.
- **Auditorías de memorización pre-deployment:** el toolkit publicado por Carlini et al. permite medir la extractability de un modelo before deployment; ejecutar esta auditoría sobre el modelo fine-tuneado con datos sensibles antes de desplegarlo es el único mecanismo para cuantificar el riesgo antes de la exposición.
- **Implicaciones para fine-tuning corporativo:** datos de fine-tuning (contratos, emails, historiales clínicos) tienen mayor riesgo de memorización que datos de pretraining de Common Crawl porque el ratio de duplicación es más alto; un empleado que accede al modelo fine-tuneado puede extraer datos a los que no debería tener acceso directamente.

La privacidad del training data no termina cuando el entrenamiento concluye: el modelo resultante puede revelar datos sensibles del corpus de entrenamiento durante su vida útil en

producción, y la única mitigación efectiva combina deduplicación agresiva antes del entrenamiento con differential privacy durante el entrenamiento.

Módulo 9 – Capítulo 06 – Sección 04

Differential privacy en fine-tuning: técnicas para proteger datos de entrenamiento sensibles

Differential Privacy (DP) es el framework matemático que proporciona la única garantía formal y cuantificable sobre la privacidad de los datos de entrenamiento: dado un parámetro epsilon (el presupuesto de privacidad), DP garantiza matemáticamente que la probabilidad de que un adversario pueda inferir si un dato específico estuvo en el training set está acotada, independientemente del poder computacional del adversario y de cuánta información tenga sobre el modelo. Esta garantía matemática es cualitativamente diferente a cualquier promesa basada en buenas prácticas: no es "hicimos nuestro mejor esfuerzo para proteger los datos" sino "matemáticamente, la información que el modelo puede revelar sobre cualquier ejemplo individual está acotada por epsilon".

DP-SGD (Differentially Private Stochastic Gradient Descent), propuesto por Abadi et al. en 2016 y popularizado por la librería Opacus de Meta (open-source, compatible con PyTorch), es el algoritmo estándar para implementar differential privacy durante el entrenamiento de modelos de deep learning. El mecanismo de DP-SGD opera en dos pasos que se aplican en cada batch del gradiente descent: primero, gradient clipping, que limita la norma del gradiente de cada ejemplo individual a un valor máximo C (por defecto $C=1.0$ en Opacus), reduciendo la influencia máxima que cualquier ejemplo individual puede tener sobre los pesos del modelo; segundo, adición de ruido gaussiano calibrado al presupuesto epsilon y a C , que introduce incertidumbre suficiente para garantizar que el modelo no puede revelar si cualquier ejemplo específico estuvo en el training set con probabilidad mayor a la garantizada por epsilon.

El trade-off fundamental de DP entre privacidad y utilidad del modelo es el aspecto más importante para el AI Engineer que debe decidir si implementarlo y con qué parámetros. Un epsilon pequeño ($\epsilon < 1.0$, considerado "privacidad fuerte") añade suficiente ruido para que la degradación en accuracy del modelo sea significativa —típicamente 3-8% de degradación para modelos medianos entrenados con datasets de fine-tuning de tamaño moderado. Un epsilon mayor ($\epsilon = 8-10$) proporciona protección mucho más débil

matemáticamente pero degrada la accuracy menos. La elección correcta de epsilon depende de la sensibilidad de los datos: para datos de salud o datos financieros de alta sensibilidad, $\epsilon < 1.0$ puede ser la única opción éticamente justificable; para datos de menor sensibilidad, $\epsilon = 8$ puede proporcionar protección suficiente con menor degradación de calidad.

La combinación de PEFT (Parameter-Efficient Fine-Tuning) con DP-SGD —específicamente LoRA (Low-Rank Adaptation) con Opacus— ha emergido como la práctica más efectiva para fine-tuning de LLMs con privacidad: LoRA reduce el número de parámetros sujetos a entrenamiento (y por tanto al ruido de DP) a menos del 1% de los parámetros totales del modelo, lo que reduce drásticamente la degradación causada por el ruido diferencial mientras mantiene la utilidad del fine-tuning. Un sistema que hace fine-tuning de Llama-3 o Mistral sobre datos de usuarios con LoRA + DP-SGD puede mantener $\epsilon = 3-5$ con degradación de calidad inferior al 2%, un trade-off operacionalmente aceptable para la mayoría de los casos de uso con datos sensibles.

Aspectos técnicos

- **DP-SGD con Opacus (Meta, open-source):** gradient clipping (norma máxima por ejemplo $C=1.0$) + adición de ruido gaussiano calibrado a epsilon y delta; compatible con la mayoría de arquitecturas de Transformers; requiere wrapping del optimizador de PyTorch con `PrivacyEngine`.
- **Presupuesto de privacidad (epsilon):** $\epsilon < 1.0$ = privacidad fuerte (recomendado para PHI o datos financieros de alta sensibilidad), $\epsilon 1-10$ = privacidad moderada (suficiente para la mayoría de aplicaciones con datos sensibles), $\epsilon > 10$ = protección matemática débil; el valor de epsilon debe documentarse en el design document del sistema.
- **LoRA + DP-SGD:** combinar fine-tuning con LoRA (solo ~0.1-1% de parámetros entrenados) con DP-SGD reduce la degradación de accuracy porque el ruido afecta menos parámetros; práctica recomendada para fine-tuning de LLMs grandes con datos sensibles.
- **Privacy accounting:** el presupuesto epsilon se "consume" con cada paso de entrenamiento; Opacus implementa privacy accounting automático con Rényi DP para calcular el epsilon total consumido; el accounting debe verificarse al final del entrenamiento antes del deployment.
- **Alternativas a DP-SGD:** federated learning con agregación segura para entrenar sobre datos distribuidos sin centralizar los datos; generación de datos sintéticos con

generadores diferencialmente privados para crear datasets que preservan propiedades estadísticas sin PII.

Nota del Arquitecto: *Differential privacy suena a "investigación académica" hasta que un regulador de GDPR pregunta cómo puedes garantizar que el modelo no puede revelar datos personales de las personas cuya información usaste para entrenarlo. En ese momento, "hicimos anonimización básica" no es una respuesta suficiente; DP es la única respuesta con respaldo matemático. He implementado DP-SGD con Opacus en producción y el trade-off real en accuracy para $\epsilon = 5-8$ con LoRA es menor de lo que los papers académicos sugieren —típicamente 1-3% en tareas de clasificación y NLU, mucho menos que el costo de un incidente de privacidad.*

Differential privacy es la única técnica que proporciona garantías matemáticas cuantificables sobre la privacidad del training data; para sistemas que procesan datos de salud, datos financieros o cualquier categoría de datos de alta sensibilidad, la implementación de DP-SGD con ϵ documentado no es opcional sino el requisito técnico mínimo que puede sostener una auditoría regulatoria de privacidad.

Módulo 9 – Capítulo 06 – Sección 05

Auditoría de acceso a datos: logs de qué datos fueron procesados por el modelo

La auditoría de acceso a datos en sistemas de IA responde a preguntas que son críticas tanto para la seguridad operacional como para el compliance regulatorio: ¿qué datos personales de qué usuarios fueron procesados por qué versión del modelo en qué momento?, ¿qué documentos del corpus RAG fueron recuperados y usados como contexto para generar una respuesta específica?, ¿qué herramientas invocó el agente con qué argumentos durante la sesión de un usuario determinado? Estas preguntas son triviales de responder en sistemas CRUD con logs de queries de base de datos, pero en sistemas de IA requieren un diseño deliberado del sistema de logging porque el "procesamiento" de un dato por un LLM no genera automáticamente trazas auditables equivalentes.

El diseño del audit log para sistemas de IA requiere identificar qué información debe registrarse para responder a los tres casos de uso principales: el análisis forense de incidentes de seguridad (¿qué ocurrió exactamente y qué datos fueron expuestos?), el compliance regulatorio proactivo (¿podemos responder a una DSAR de GDPR dentro de 30 días?, ¿podemos demostrar que los controles de HIPAA estuvieron activos?), y el análisis de riesgo operacional (¿qué usuarios intentan ataques de jailbreak?, ¿qué documentos RAG se recuperan más frecuentemente ante queries relacionadas con temas sensibles?). El schema del audit log debe ser suficientemente rico para soportar todos estos casos de uso, pero también suficientemente disciplinado en la retención de texto completo para no crear él mismo un riesgo de privacidad.

La granularidad del tracking de documentos RAG es el elemento más específico de los sistemas de IA. Cada inferencia que usa RAG debe registrar los IDs únicos de los documentos recuperados del vectorstore, los scores de similitud con la query, y si el documento fue efectivamente incluido en el contexto enviado al modelo (top-k puede recuperar más documentos de los que caben en el contexto). Este nivel de granularidad permite responder preguntas como "¿los documentos de este usuario fueron recuperados como contexto para respuestas a otros usuarios?" —pregunta crítica para detectar cross-user contamination en

RAG compartido y para responder DSARs de GDPR que pregunten qué uso se hizo de los datos del sujeto.

Los sistemas que procesan PHI bajo HIPAA tienen un requisito adicional: los audit logs de acceso a PHI deben mantenerse por mínimo seis años, y deben registrar no solo qué datos fueron accedidos sino también el "propósito" del acceso —qué tarea estaba realizando el usuario que generó el acceso a esos datos. Para sistemas de IA donde el "propósito" no es siempre explícito (el usuario hizo una pregunta y el RAG recuperó documentos que contenían PHI sin que el usuario lo solicitara específicamente), el log debe registrar suficiente contexto para que un auditor pueda entender el flujo: la query del usuario, los documentos recuperados, y si el modelo utilizó información de esos documentos en su respuesta.

Aspectos técnicos

- **Schema del security event para sistemas de IA:** `{event_id, timestamp, user_id, session_id, request_id, model_id, model_version, system_prompt_hash, input_token_count, input_hash, retrieved_document_ids[], retrieved_document_scores[], tool_calls[{name, args_hash, result_hash}], output_hash, input_safety_score, output_safety_score, latency_ms}` con texto completo de input/output cifrado por usuario en campo separado.
- **Inmutabilidad del audit log:** S3 Object Lock en COMPLIANCE mode (AWS), Azure Immutable Blob Storage, o WORM storage equivalente; los logs de auditoría deben ser write-once para ser válidos como evidencia regulatoria y forense.
- **DSAR compliance automation (GDPR):** el audit log debe ser consultable por `user_id` para exportar todos los eventos que involucran datos de un usuario específico dentro de los 30 días que el GDPR permite; la estructura del log debe diseñarse con esta query en mente desde el inicio.
- **Borrado selectivo para el derecho al olvido:** implementar borrado mediante cifrado por usuario: cada usuario tiene una clave de cifrado única para el texto de sus inputs/outputs; el "borrado" consiste en descartar la clave, haciendo ilegible el contenido aunque el registro del evento permanezca.
- **Tracking de documentos RAG:** registrar `retrieved_document_ids[]` con scores de similitud permite responder si los documentos de un usuario fueron usados como contexto para respuestas a otros usuarios — crítico para detectar cross-user contamination y para DSARs de GDPR.

El audit log de un sistema de IA es el instrumento de análisis forense más crítico ante un incidente: sin los IDs de documentos RAG recuperados, los argumentos de tool calls, y los scores de clasificadores de seguridad, es imposible reconstruir si el sistema fue comprometido y cómo.

Módulo 9 – Capítulo 06 – Sección 06

Cierre: la privacidad en IA es una propiedad del sistema, no solo del almacenamiento

La privacidad en sistemas de IA trasciende el paradigma clásico de "cifrar los datos en reposo y en tránsito y habremos cumplido". Un sistema puede tener todas sus bases de datos cifradas con AES-256, todos sus canales protegidos con TLS 1.3, y haber firmado todos los Business Associate Agreements requeridos —y aun así violar sistemáticamente la privacidad de sus usuarios si el modelo reproduce datos personales memorizados de su corpus de entrenamiento, si el historial de conversación es usado para fine-tuning sin consentimiento informado, o si documentos de un usuario son recuperados por RAG como contexto para las respuestas de otro usuario sin controles de segmentación. El cifrado protege los datos mientras están en reposo o en tránsito; no protege la información que el modelo extrajo de esos datos durante el entrenamiento y que ahora reside en sus pesos.

La privacidad en IA es una propiedad emergente del sistema completo —del diseño del pipeline de datos, del proceso de entrenamiento, de la arquitectura del sistema de inferencia, de los controles de acceso al vectorstore, y del diseño del sistema de logging— que debe planificarse intencionalmente desde la arquitectura. El GDPR Art. 25 ("Privacy by Design and by Default") y el EU AI Act formalizan este enfoque sistémico como requisito legal para sistemas que procesan datos personales, pero la justificación técnica existe independientemente del marco regulatorio: la privacidad que no se diseña en el sistema no puede añadirse efectivamente después del deployment sin un rediseño sustancial.

Las cinco dimensiones de privacidad que este capítulo estableció —detección y anonimización de PII en todas las capas, cifrado integral de datos en tránsito y en reposo incluyendo índices vectoriales, gestión del riesgo de memorización de training data, differential privacy durante el fine-tuning con datos sensibles, y auditoría granular de acceso a datos— son un sistema de controles interdependientes. La ausencia de cualquiera de ellos crea brechas que los demás no pueden compensar: la differential privacy durante el entrenamiento no sirve de nada si el corpus de fine-tuning no fue deduplicado y contiene el mismo dato repetido cientos de veces;

el audit log de acceso a documentos RAG no protege la privacidad si esos documentos fueron indexados sin pasar por detección de PII.

Conceptos clave del capítulo

- **PII en todas las capas:** detección automática con Microsoft Presidio antes de indexar en RAG, anonimización/pseudonimización del corpus de fine-tuning, validación de outputs para prevenir exposición de PII memorada o del contexto del usuario.
- **Cifrado integral:** TLS 1.3 en tránsito para todas las APIs y servicios internos, AES-256 con claves KMS para vectorstores y artefactos de modelo en reposo, logs de inferencia como datos sensibles de primera clase con cifrado por usuario.
- **Memorización de training data:** riesgo cuantificable y documentado (Carlini et al., Nasr et al.); mitigado con deduplicación agresiva del corpus antes del entrenamiento y differential privacy (DP-SGD con Opacus) durante el fine-tuning.
- **Differential privacy:** garantía matemática con trade-off utilidad/privacidad calibrado al epsilon; LoRA + DP-SGD como combinación práctica para fine-tuning con datos sensibles con degradación de accuracy controlada.
- **Audit log para compliance:** registro inmutable de model_id, user_id, retrieved_document_ids[], tool_calls[], y safety scores; retention con cifrado por usuario y borrado selectivo por descarte de clave para el derecho al olvido del GDPR.

"Privacy is not a feature you add; it's an architecture you design." — Ann Cavoukian, inventora del marco "Privacy by Design" y ex Comisionada de Información y Privacidad de Ontario, Canadá, cuyo principio de privacidad desde el diseño es hoy requisito legal en el GDPR Art. 25 y una necesidad técnica real en sistemas de IA que procesan datos personales a través de múltiples capas de procesamiento.